

Clase 006 — Línea de comandos Linux avanzada: grep, sed, awk, pipes y procesos

Parte: **0 — Fundamentos y prerequisites** · Fuente: *Shotts, The Linux Command Line (No Starch Press)*
⌚ Duración estimada: **110 min** · Nivel: **Fundamentos**

Objetivo

Convertir la terminal en tu herramienta más rápida para procesar texto, logs y salidas de herramientas. Al terminar podrás encadenar comandos con pipes, filtrar con `grep`, transformar con `sed`, extraer campos con `awk` y controlar procesos, habilidades imprescindibles para el análisis de logs y la automatización.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Construir** tuberías (pipes) que combinen varios comandos.
- 2. **Filtrar** texto con `grep` y expresiones regulares básicas.
- 3. **Transformar** flujos con `sed` (sustitución, borrado, rangos).
- 4. **Extraer** y agregar campos con `awk`.
- 5. **Gestionar** procesos: listar, priorizar, señales y jobs.

Temas

#	Tema	Por qué importa
1	Streams y redirección	stdin/stdout/stderr son la base de todo
2	Pipes	Componer herramientas pequeñas resuelve problemas grandes
3	<code>grep</code>	Buscar patrones en logs y código
4	<code>sed</code>	Editar flujos sin abrir un editor
5	<code>awk</code>	Procesar datos por columnas
6	Orden y unicidad	<code>sort</code> , <code>uniq</code> , <code>cut</code> , <code>tr</code> , <code>wc</code>
7	Procesos	<code>ps</code> , <code>top</code> , <code>kill</code> , señales
8	Jobs y background	<code>&</code> , <code>jobs</code> , <code>fg</code> , <code>nohup</code>

Definiciones y características

- **stdin/stdout/stderr**: canales estándar (0, 1, 2). Clave: `2>` redirige errores; `>` y `>>` escriben/anexan.
- **Pipe (|)**: conecta la salida de un comando con la entrada del siguiente. Clave: filosofía Unix de herramientas componibles.

- **grep**: filtra líneas que coinciden con un patrón. Clave: `-r` recursivo, `-i` sin distinción de mayúsculas, `-E` regex extendida.
- **sed**: editor de flujo orientado a líneas. Clave: `s/patrón/reemplazo/g` es su uso más común.
- **awk**: lenguaje para procesar texto por campos. Clave: `$1`, `$2` ... son las columnas; `NR` el número de línea.
- **Señal**: mensaje al proceso (SIGTERM, SIGKILL, SIGHUP). Clave: `kill -9` es forzado y no permite limpieza.

Herramientas y preparación

Todo viene de serie en Linux: `grep`, `sed`, `awk` (gawk), `sort`, `uniq`, `cut`, `tr`, `wc`, `ps`, `top` / `htop`, `kill`. Descarga un log de ejemplo (por ejemplo `/var/log/auth.log` o el `access.log` de un servidor web) para practicar sobre datos reales.

Laboratorio guiado

1. Redirección y streams:

```
bash ls /noexiste /etc 1>salida.txt 2>errores.txt ; cat errores.txt
```

2. Pipes básicas. Cuenta cuántos usuarios hay en el sistema:

```
bash cut -d: -f1 /etc/passwd | sort | wc -l
```

3. grep en logs. Busca intentos de login fallidos:

```
bash grep -i "failed password" /var/log/auth.log | head
```

4. Extraer IPs con awk. De esos fallos, saca la IP atacante (ajusta el campo):

```
bash grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -nr
```

Esto da un *top* de IPs por número de intentos. 5. **sed para limpiar**. Elimina comentarios y líneas vacías de un config:

```
bash sed -e 's/#.*//' -e '/^\s*$/d' /etc/ssh/sshd_config
```

6. Procesos. Encuentra el proceso que más CPU consume y su PID:

```
bash ps aux --sort=-%cpu | head -5
```

7. Señales. Lanza un proceso en background y térmalo limpiamente:

```
bash sleep 300 & jobs ; kill %1
```

Ejercicios

1. A partir de un `access.log`, obtén el top 10 de IPs con más peticiones.
2. Cuenta cuántas peticiones devolvieron código 404 usando `awk`.
3. Con `sed`, cambia todas las apariciones de `http://` por `https://` en un archivo.
4. Extrae solo los nombres de usuario con shell `/bin/bash` de `/etc/passwd`.

5. Muestra los 5 procesos que más memoria consumen, con usuario y PID.
6. Combina `grep`, `sort` y `uniq -c` para contar user-agents distintos en un log web.

Reto verificable

Escribe una única línea (one-liner) que, a partir de un log de acceso web, produzca un informe ordenado del top 10 de IPs por número de peticiones **excluyendo** las de tu propia red interna. Documenta cada comando de la tubería.

Criterio de aceptación: el one-liner ejecuta sin errores sobre un `access.log` real y su salida es una lista de exactamente hasta 10 IPs con su conteo, ordenadas de mayor a menor y sin incluir la subred interna filtrada. Otra persona puede explicar qué hace cada segmento del pipe.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>grep</code> no encuentra lo que ves en pantalla	Regex mal escrita o mayúsculas. Prueba <code>-i</code> y escapa caracteres especiales.
<code>awk</code> imprime campo vacío	El separador no es el esperado. Usa <code>-F</code> para fijar el delimitador (<code>-F:</code>).
<code>sed -i</code> corrompe o vacía el archivo	Editaste in-place sin copia. Prueba primero sin <code>-i</code> y guarda backup con <code>-i.bak</code> .
<code>kill</code> no mata el proceso	Ignora SIGTERM. Usa <code>kill -9</code> (SIGKILL) como último recurso.
El pipe pierde los errores	<code>stderr</code> no viaja por el pipe; redirígelo con <code>2>&1</code> si lo necesitas.

Preguntas frecuentes

 **¿grep, sed o awk?** `grep` filtra, `sed` transforma línea a línea, `awk` procesa por columnas y agrega. Para contar/estadística por campos, `awk`; para sustituir texto, `sed`; para buscar, `grep`.

 **¿kill -9 siempre es mejor?** No. SIGKILL no deja al proceso limpiar (archivos temporales, sockets). Usa primero SIGTERM; reserva `-9` para procesos colgados.

 **¿Por qué mi one-liner es lento con archivos enormes?** Ordenar y varias pasadas cuestan. Filtra pronto con `grep`, reduce datos antes de `sort`, y considera `awk` para hacerlo todo en una pasada.

 **¿Necesito aprender regex a fondo?** Lo básico ya multiplica tu productividad; en la Clase 019 profundizamos en expresiones regulares para logs y datos.

Referencias

- William Shotts, *The Linux Command Line* — <https://linuxcommand.org/tlcl.php>
- `man 1 grep`, `man 1 sed`, `man 1 awk`
- GNU Awk User's Guide — <https://www.gnu.org/software/gawk/manual/>
- `man 7 signal`

Siguiente clase

Clase 007 - Bash scripting para tareas de seguridad